



UNIVERSIDAD
DE GRANADA

TRABAJO FIN DE GRADO
INGENIERÍA INFORMÁTICA

Estudio e implementación paralela de métodos semiimplícitos multipaso

Resolución de Ecuaciones de Advección-Reacción-Difusión

Autor

José Manuel Navarro Cuartero

Director

José Miguel Mantas Ruiz



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, Septiembre de 2026

**Estudio e implementación paralela de métodos
semiimplícitos multipaso:
Resolución de Ecuaciones de Advección-Reacción-Difusión**

José Manuel Navarro Cuartero

Palabras clave: palabra_clave1, palabra_clave2, palabra_clave3,

Resumen

Poner aquí el resumen.

Project Title: Project Subtitle

First name, Family name (student)

Keywords: Keyword1, Keyword2, Keyword3,

Abstract

Write here the abstract in English.

Yo, **José Manuel Navarro Cuartero**, alumno de la titulación **Grado en Ingeniería Informática** de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 75896777C, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: José Manuel Navarro Cuartero

Granada a X de Septiembre de 2026.

D. **José Miguel Mantas Ruiz**, Profesor del Área de XXXX del Departamento de Lenguajes y Sistemas Informáticos de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado **Estudio e implementación paralela de métodos semiimplícitos multipaso, Resolución de Ecuaciones de Advección-Reacción-Difusión**, ha sido realizado bajo su supervisión por **José Manuel Navarro Cuartero**, y autorizo la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expide y firma el presente informe en Granada a X de Septiembre de 2026.

El director:

José Miguel Mantas Ruiz

Agradecimientos

Poner aquí agradecimientos...

Índice

Introducción	1
Conceptos previos	3
Métodos numéricos y EDOs	3
Tipos de métodos numéricos	4
Método de Euler	5
Métodos de Resolución	7
Planteamiento	7
Runge Kutta	7
Adams-Bashford	8
Adams-Bashford-Moulton	8
Problemas	9
SimpleAdvDiff	9
AdvDiff1d	9
Brusselator1d	9
Brusselator2d	9
Paralelismo	10
Secuencial	10
OpenMP	10
CUDA	10

Introducción

Los métodos numéricos son un conjunto de técnicas y algoritmos matemáticos diseñados para encontrar soluciones aproximadas a problemas que, en muchos casos, no pueden resolverse de manera exacta mediante el análisis o su resolución sería muy costosa. Se utilizan asiduamente en diversos campos de la ciencia, ingeniería, economía y muchas otras disciplinas para resolver ecuaciones diferenciales, integrales, sistemas de ecuaciones, aproximación e interpolación, integración numérica y más. Es en el primero de estos casos en el que nos centraremos. Si bien es cierto que existen diversas técnicas para la resolución de ecuaciones diferenciales ordinarias y de sistemas de ecuaciones diferenciales, éstas solo se pueden aplicar cuando dichas ecuaciones se ajustan a un patrón concreto.

En el mundo de los métodos numéricos hay infinitud de formas de resolver las EDOs, y tantas o más aplicaciones de las mismas a problemas del mundo real. Escoger el método correcto tiene que ver tanto con la exactitud de la solución, como con su coste, de potencia de computación y tiempo.

Es en este último punto en el que nos centraremos en los siguientes capítulos, ya que si bien para problemas pequeños, algo tan simple como el método de Euler explícito podría bastarnos, posteriormente plantearemos problemas mucho más complejos, que requerirán soluciones que estén a la altura. No se trata por tanto solamente de hallar una solución, sino de hacerlo rápida y eficientemente.

Para esto, partiremos de 4 problemas cuya implementación secuencial y soluciones esperadas ya conocemos de forma previa, y aplicaremos para su resolución 3 métodos numéricos distintos, que a su vez implementaremos con dos tecnologías para paralelizar distintas, OpenMP y CUDA, es decir, paralelismo a nivel de CPU y GPU.

Nuestro objetivo será implementar de forma fiel y exacta tanto los problemas como los métodos de resolución, para posteriormente poder comparar las ganancias en tiempo de ejecución al pasar de una tecnología a otra.

La idea detrás de todo esto es relativamente simple, un algoritmo que utilice 4 o 16 hebras será mejor que uno secuencial. Y uno que utilice 256 será mejor que todos ellos. Aunque la realidad no siempre va de la mano de nuestras ideas, buscaremos conseguir las mejores ganancias aprovechando al máximo todas las herramientas que ponen a nuestra disposición tanto CUDA

como OpenMP. Haremos por tanto serie de mediciones y buscaremos probar nuestra hipótesis.

Conceptos previos

Métodos numéricos y EDOs

Una ecuación diferencial es aquella que relaciona una función ($f(x)$ ó y), sus variables (x) y sus derivadas ($f'(x)$, dy/dx ó y'). Para resolver una ecuación diferencial se debe encontrar una función que satisfaga dicha ecuación. Dentro de las ecuaciones diferenciales nos encontramos las ecuaciones diferenciales ordinarias, EDOs, que contienen derivadas respecto a una sola variable independiente. Más adelante nos centraremos en este tipo en particular. En oposición a éstas estarían las ecuaciones diferenciales parciales, en las que la función $f(x)$ depende de más variables independientes. [Molero et al., 2007]

El orden de una ecuación diferencial equivale al orden de la mayor derivada que aparece en la ecuación. Además, será lineal cuando la variable dependiente y todas sus derivadas son de primer grado (no puede haber $f(x)^2$), y los coeficientes de la variable dependiente y sus derivadas dependen de la variable independiente.

A modo de ejemplo simple podríamos considerar una ecuación diferencial ordinaria de primer orden como:

$$\frac{dy}{dt} = -2y, y(0) = 1$$

Donde además de la EDO, se nos está dando su valor inicial. En este caso en particular, sabemos de forma analítica que su solución exacta es $y(t) = e^{-2t}$, y podríamos por tanto calcular cualquier y_n que deseemos, pero si por cualquier razón no pudiéramos resolverla analíticamente deberíamos recurrir a los métodos numéricos. A través de ellos, y partiendo del valor y_n que se nos proporciona, calcularíamos de forma iterativa y_{n+1} hasta llegar al valor n que busquemos.

Podríamos utilizar, por ejemplo, el método de Euler, del que hablaremos más adelante. Antes de ello podemos entrar en algo más de detalle respecto a algunos otros conceptos que iremos usando.

Tipos de métodos numéricos

Podemos dividir a los métodos en tres grupos, explícitos, implícitos y semiimplícitos. La división se hará en función de cómo se consigue el siguiente paso, es decir, de cómo obtenemos y_{n+1} a partir de y_n ; de si aparecen valores futuros o no en la fórmula que lo calcula. Tendremos como expresión general la siguiente: [Molero et al., 2007]

$$y_{n+1} = y_n + \Delta t \cdot \Phi(x_n, y_n, y_{n+1}, \Delta t) \quad (1)$$

En la cual será la función incremento Φ la que dirima a cuál de los grupos pertenece el método con el que estemos trabajando.

Los métodos explícitos son aquellos en los que siguiente término se puede obtener directamente, sólo haciendo cálculos con datos de los que ya se dispone. Dicho de otra forma, son aquellos en los que y_{n+1} no aparece en Φ , sólo aparece en la parte izquierda de la ecuación. Entre ellos nos encontraremos el Euler explícito, Runge-Kutta, o Adams-Bashford, los tres los veremos posteriormente. Los métodos explícitos suelen ser más fáciles de implementar, ya que no requieren resolver ecuaciones, y suelen ser más rápidos. Por otro lado, tienden a ser menos estables, especialmente en problemas rígidos (*stiff*).

Al otro lado del espectro nos encontraríamos a los métodos implícitos, que son aquellos en los que y_{n+1} sí aparece en la parte de la derecha de la igualdad, lo cuál nos obligaría a resolver una ecuación que podrá ser, o no, lineal. Estos métodos son muy estables, y son más adecuados para resolver problemas rígidos (*stiff*). Como parte negativa tienen que requieren resolver ecuaciones en cada paso, y a veces de forma iterativa, lo que los acaba haciendo mucho más costosos computacionalmente. Entre ellos nos encontramos algunos métodos como el Euler implícito, Adams-Moulton, o los métodos BDF.

Como tercer grupo tendríamos los métodos semiimplícitos. Éstos representan una especie de "solución" a los diferentes problemas que traen consigo los métodos explícitos e implícitos. En este grupo, aunque existe una parte implícita, ésta es más simple, ya que o bien la ecuación a resolver es lineal, o bien se separa la parte rígida para tratarla implícitamente y el resto se trata de forma explícita. Esto es útil cuando el problema tiene términos rígidos (*stiff*) como la difusión, que deben tratarse implícitamente para que el método sea estable incluso con pasos de tiempo grandes, y términos suaves o de comportamiento hiperbólico como la advección, que se pueden tratar explícitamente para evitar resolver sistemas grandes o no lineales. Los métodos semiimplícitos representan un compromiso entre la estabilidad de los explícitos y la velocidad de los implícitos. Como hemos mencionado, podremos dividir a la EDO en dos partes:

$$y' = f(x, y) = f_E(x, y) + f_I(x, y) \quad (2)$$

En este ejemplo f_E es la parte explícita, suave (no *stiff*), y f_I es la parte que queremos tratar implícitamente. Algunos métodos semiimplícitos son los métodos IMEX (*implicit-explicit*), los métodos linealmente implícitos Runge-Kutta (*LIRK*), o el método Adams-Bashford-Moulton (AB-AM predictor-corrector). Es con este último con el que trabajaremos.

Otra categorización con la que trabajaremos serán los métodos de un solo paso, y los métodos multipaso. En los métodos de un solo paso, para obtener el siguiente valor con el que se tratará, el siguiente paso, sólo se utiliza información relativa al paso anterior. Es decir, como podemos ver en 1, para obtener y_{n+1} sólo se utilizaría y_n .

Sin embargo, los métodos multipaso usan varios valores anteriores de la solución para calcular el siguiente paso en el tiempo, en lugar de basarse solo en el valor actual. Es decir, que para obtener y_{n+1} se podrían tomar pasos intermedios que posteriormente se desechen (como en Runge-Kutta), o utilizar pasos más antiguos como y_{n-1} o y_{n-2} . Para problemas de gran tamaño, los métodos multipaso permiten alcanzar resultados con gran precisión y de una forma más eficiente que los métodos de un paso como el de Euler o el de Runge-Kutta.

Método de Euler

Como hemos comentado, el método de Euler es uno de los métodos numéricos más simples de los que disponemos para la resolución de EDOs. Se trata de un método explícito de un solo paso que se construye a partir de una sencilla idea geométrica: aproximar el valor de la solución en cada nodo por el valor que proporciona la recta tangente a la solución trazada desde el punto correspondiente al nodo anterior. [Gallardo, 2018]

Emplearíamos la siguiente fórmula:

$$y_{n+1} = y_n + \Delta t \cdot f(t_n, y_n) \quad (3)$$

Aplicando la misma al ejemplo de valor inicial que mencionamos antes, $\frac{dy}{dt} = -2y$, $y(0) = 1$, sabemos que y_n corresponde a la aproximación numérica de $y(t_n)$, y Δt representa la variación en el tiempo paso a paso que estemos considerando. Además, en nuestro caso, $f(t_n, y_n)$ sería $-2 \cdot y_n$, y se nos da el valor inicial $t_0 = 0.0$. Por tanto, tomando estos valores, con un $\Delta t = 0.1$, y $t_f = 1.0$, calcularemos los sucesivos valores para $t = 0.1, 0.2, \dots, 1.0$.

El primer paso sería por tanto, partiendo de los valores iniciales, calcular el $t = 0.1$, a saber, $y_1 = y_0 + \Delta t \cdot f(t_0, y_0) = 1 + 0.1 \cdot (-2 \cdot 1) = 0.8$. Si esto lo repitiéramos de forma sucesiva obtendríamos los siguientes resultados:

Paso n	t_n	$y_n(\text{Euler})$	$y_n(\text{Exacta})$	Diferencia	Dif/y_n
0	0.0	Valor Inicial = 1.0		-	-
1	0.1	0.8	0.81873	0.01873	0.022
2	0.2	0.64	0.67032	0.03032	0.045
3	0.3	0.512	0.54881	0.03681	0.067
4	0.4	0.4096	0.44933	0.03973	0.088
5	0.5	0.32768	0.36788	0.04020	0.109
6	0.6	0.26214	0.30119	0.03905	0.130
7	0.7	0.20972	0.24660	0.03688	0.150
8	0.8	0.16777	0.20190	0.03412	0.169
9	0.9	0.13422	0.16530	0.03108	0.188
10	1.0	0.10737	0.13533	0.02796	0.207

Cuadro 1: Comparativa entre Euler y valor real

Como podemos observar en la tabla, el error que cometemos en la aproximación se mantiene relativamente estable en términos absolutos, pero porcentualmente hablando respecto al valor real aumenta con cada iteración. Esto nos indica que para este problema, quizás el método que hemos utilizado o el Δt que hemos considerado no son los más indicados.

Métodos de Resolución

Planteamiento

Para resolver los diferentes problemas hemos implementado tres métodos distintos, a saber: Runge-Kutta, que es explícito y utiliza pasos intermedios ($K1, K2$) para obtener los siguientes pasos; Adams-Bashford, que también es explícito y multipaso, pero utiliza pasos antiguos (como f_{n-2}); y Adams-Bashford-Moulton, un predictor-corrector semiimplícito basado en Adams-Moulton (inherentemente implícito) que utiliza Adams-Bashford para

Runge Kutta

Los métodos de Runge-Kutta se utilizan para resolver ecuaciones diferenciales no lineales, cuando encontrar una solución exacta es muy difícil o imposible. En el método de Runge-Kutta de orden 4 resolveremos diversos pasos que nos servirán para estimar de forma muy aproximada el resultado. Partiendo de un valor inicial $dy/dt = f(t, y)$, $y(t_0) = y_0$, y de un valor de salto $h > 0$, calcularemos iterativamente los valores tal que:

$$\begin{aligned}y_{n+1} &= y_n + \frac{h}{6} \cdot (k_1 + 2k_2 + 2k_3 + k_4) \\k_1 &= f(t_n, y_n) \\k_2 &= f\left(t_n + \frac{h}{2}, y_n + h \cdot \frac{k_1}{2}\right) \\k_3 &= f\left(t_n + \frac{h}{2}, y_n + h \cdot \frac{k_2}{2}\right) \\k_4 &= f(t_n + h, y_n + h \cdot k_3)\end{aligned}\tag{4}$$

Aplicado en un contexto de programación, tendremos una clase *RungeKutta* con una función principal que aplicará el cálculo y algunas auxiliares que se explicarán más adelante. Dicha función tendrá el siguiente diseño, en el que se harán diversas llamadas a la evaluación de la derivada de cada problema en cuestión (aquí lo llamaremos *feval*):

Algorithm 1 RungeKutta Orden 4

```
 $t_n \leftarrow t_0$   
while  $t_n \leq t_f$  do  
   $K1 \leftarrow feval(t_n, Y_n)$   
   $K2 \leftarrow feval(t_n + h/2, Y_n + K1 \cdot h/2)$   
   $K3 \leftarrow feval(t_n + h/2, Y_n + K2 \cdot h/2)$   
   $K4 \leftarrow feval(t_n + h, Y_n + K3 \cdot h)$   
   $Y_{n+1} \leftarrow Y_n + h/6 \cdot (K1 + 2 \cdot K2 + 2 \cdot K3 + K4)$   
   $t_n + = h$   
end while
```

Como podemos ver, la aplicación del algoritmo es relativamente simple, se trabaja con 4 vectores auxiliares en los que hacemos los cálculos intermedios, que luego se suman. Es en estas operaciones, las evaluaciones de cada miembro, las sumas, y las multiplicaciones de cada vector, en las que podremos entrar a trabajar el paralelismo para mejorar la eficiencia de nuestro código.

Adams-Bashford

Adams-Bashford-Moulton

Problemas

SimpleAdvDiff

En primero de los programas con los que trataremos será este SimpleAdvDiff, que representa un modelo simple de Advección-Difusión en 1D. En este problema trabajaremos con un vector que estará compuesto de dos partes, una parte rígida y una no rígida, que se sumarán para dar los valores de cada una de las celdas del vector.

AdvDiff1d

Brusselator1d

Brusselator2d

Paralelismo

Secuencial

OpenMP

CUDA

Mediciones

Bibliografía

- [Gallardo, 2018] Gallardo, J. M. (2018). *Análisis Numérico de Ecuaciones Diferenciales*. Universidad de Málaga.
- [Molero et al., 2007] Molero, M., Alcaide, A., Menarguez, T., and Luis, S. (2007). *Análisis Matemático para Ingeniería*. Pearson Educación.

Glosario de Tablas

1.	Comparativa entre Euler y valor real	6
----	--	---

Glosario de Imágenes